data_in [31..0]

Apply (or not) the sign extension in Load operation for bytes and halfword
EXT1
Extension
addr
data_in
data_out
SgnE_8
SgnE_16
PT
ZrE_8
ZrE_16
Sgn8
Sgn16
PT
Zr8
Zr16
out_ext [31..0]

RF_IN
ldALU
ldMEM
ldPC
s0
s1
s2

RF1
RF
data_in
read_addr_1
read_addr_2
write_addr
chip_en
write_en_n
clk
rst_n
data_out_1
data_out_2
data_in_RF [31..0]
Raddr_1_RF [4..0]
Raddr_2_RF [4..0]
Waddr_RF [4..0]
ce
we_n
clk
rst_n
data_out_1_RF [31..0]
data_out_2_RF [31..0]

A
selRS1
selPC
in_a_ALU [31..0]

ALU1
ALU
in_a
in_b
opcode
flags
out
ALU_op [3..0]
flags_ALU [2..0]
out_ALU [31..0]

B
selRS2
selImm
in_b_ALU [31..0]

DEC1
Decoder
Based on the Type recived unpack the instruction and provide the correct data
data_in
RS1
RS2
RD
IMM
Imm [31..0]
For U-type we provide the IMM already shifted
Type_R
Type_I
Type_S
Type_SB
Type_U
Type_UJ

IR
loadIR
load
d_in
d_out
clk
rst_n
d_out_IR[31..0]
inst_out [31..0]

AM
selPC_addr
selALU_addr
addr_MEM [31..0]

SE1
Store_Encoder
From the two LSBs of the addr and on the type of data we want to write (Byte, Half Word, Word), activate the proper bits of write_en
addr
out
SB
SH
SW
write_en [3..0]
addr [31..0]
ld_SB
ld_SH
ld_SW

DA1
Data_Aligner
Transform my data (Byte, Half Word, Word), in order to have a word ready to be write in memory for every possible write_en. In particular, it makes copy in all the position.
data_in
data_out
data_out [31..0]

PC_IN
PC
d_in_PC [31..0]
d_out_PC [31..0]
loadPC
clk
rst_n

INC
+4
in
out
32'b11
1'b1

JC
selJump
selImm_PC
selINC_PC
selNZero_cond
selZero_cond